1. Synchronous JavaScript: As the name suggests synchronous means to be in a sequence, i.e. every statement of the code gets executed one by one. So, basically a statement has to wait for the earlier statement to get executed.

Let us understand this with the help of an example.

Example:

```
document.write("Hi"); // First
document.write("<br>");

document.write("Mayukh") ; // Second
document.write("<br>");

document.write("How are you"); // Third
```

Output: Hi Mayukh How are you

Asynchronous JavaScript: Asynchronous code allows the program to be executed immediately where the synchronous code will block further execution of the remaining code until it finishes the current one. This may not look like a big problem but when you see it in a bigger picture you realize that it may lead to delaying the User Interface.

Let us see the example how Asynchronous JavaScript runs.

```
document.write("Hi");
document.write("<br>");

setTimeout(() => {
  document.write("Let us see what happens");
}, 2000);

document.write("<br>");
document.write("End");
document.write("<br>");
```

Output: Hi End Let us see what happens

If you are using any of these, you are writing asynchronous code.
setTimeout, setInterval, promises, fetch, axios, XMLHttpRequest.

2. Is javascript asynchronous?

No, JS is not asynchronous but you can manipulate JavaScript to behave in an asynchronous way.

Basically we have two stacks main stack and side stack(Web API. Not provide by JS or V8 Engine but by browser). And our code starts executing and whenever we reach asynchronous code, it
is moved to side stack and is executed there.

In the meantime, main stack executes further code. Whenever asynchronous code is executed it is send to the task queue, and then processed through "event loop".

Event loop sends code to main stack only when stack is empty.

This doesn't stops your code and can run further without waiting for the asychronous code to be executed. Hence without any delay.

Example:

```
console.log("hey1);
setTimeout(function() {
   console.log("hey2");
}, 0);
console.log("hey3");
```

Output: hey1 hey3 hey2

This is the great solution, but it leaves a little something to be desired.

Because you can’t predict exactly when function C will resolve, you have to nest all dependent functions
within it. This gets messy fast, and leads to the infamous callback hell that no one wants to deal with. It was this environment that inspired the promise.
When there are too many functions dependent on each other, all these functions will go on WEB API. And all these functions will get queued and will take a lot of time. For resolving
this, promises came into existence.

3. What are promises?
With a promise, instead of bundling all dependencies into one code block and sending the entire thing off to the browser, we’re able to separate them out.

We can send the asynchronous callback (Function C) to the browser, and use .then() to hold all other dependencies (E, F and G) aside, running them only once Function C returns and runs.

This allows us to code in a more modular, readable way, while still gaining the benefits of asynchronous programming.

Example:

```
var ans = new Promise(res, rej) {
            return res("jflksdj");
        }

ans.then(function(data) {
      console.log(data);
}.catch {
      console.log("error");
}
```

4. What are async and await?
Promises were fantastic — so fantastic, in fact, that ES6 brought them into the language as a standard. Using promises still left asynchronous code looking and feeling slightly wonky,
so we now have beautiful and stunning Async/Await to help us out!

The async function declaration defines an asynchronous function, which returns an AsyncFunction object. An asynchronous function is a function which operates asynchronously via the
event loop, using an implicit Promise to return its result. But the syntax and structure of your code using async functions is much more like using standard synchronous functions.

Example:

```
// Promise
function abcd() {
      fetch(`url`).then(function(raw_data) {
            return raw.json();
      })
      .then(function(data) {
            console.log(data);
      });
}

abcd();

// Async and Await
async function abcd() {
      let raw_data = await fetch(`url`);
      let ans = await raw_data.json();

      console.log(ans);
}
```

5. Why to use promise rather than async and await?
   Async/Await allows you to:

   1. Continue using promises.
   2. Write asynchronous code that looks and feels synchronous.
   3. Cleans up your syntax and makes your code more human-readable.

6. Five real word uses of async/await?
   1. In node while interacting with db.
   2. While using fetch in react to give a call to backend.
   3. setTimeout
   4. setInterval

7. Concurrency: In JS both sync and async code runs simultaneously, this is only known as concurrency.
   Parallelism: Parallelism in JavaScript is an exciting concept that enables true concurrent execution of tasks, even in a primarily single-threaded language.

   Concurrency refers to performing different tasks simultaneously with differing goals, while parallelism pertains to various parts of the program executing one task simultaneously.
   Parallelism is a type of computation used to make programs run faster.

   Throttling:
      Throttling or sometimes also called throttle function is a practice used in websites. Throttling is used to call a function after every millisecond or a particular interval of time only the first click is executed immediately.

      Let’s see, what will happen if the throttle function is not present on the web page. Then the number of times a button is clicked the function will be called the same number of times. Consider the example.

      Without throttling Function: In this code suppose the button is clicked 500 times then the function will be clicked 500 times this is controlled by a throttling function.

      With Throttling function: In this code, if a user continues to click the button every click is executed after 1500ms except the first one which is executed as soon as the user clicks on the button.